

```
def map[S,A,B] (a: S => (A,S)) (f: A => B): S => (B,S)
```

Changing this signature doesn't require modifying the implementation of `map`! The more general signature was there all along; we just didn't see it.

We should then come up with a more general type than `Rand`, for handling any type of state:

```
type State[S,+A] = S => (A,S)
```

Here `State` is short for *computation that carries some state along, or state action, state transition, or even statement* (see the next section). We might want to write it as its own class, wrapping the underlying function like this:

```
case class State[S,+A] (run: S => (A,S))
```

The representation doesn't matter too much. What's important is that we have a single, general-purpose type, and using this type we can write general-purpose functions for capturing common patterns of stateful programs.

We can now just make `Rand` a type alias for `State`:

```
type Rand[A] = State[RNG, A]
```



EXERCISE 6.10

Generalize the functions `unit`, `map`, `map2`, `flatMap`, and `sequence`. Add them as methods on the `State` case class where possible. Otherwise you should put them in a `State` companion object.

The functions we've written here capture only a few of the most common patterns. As you write more functional code, you'll likely encounter other patterns and discover other functions to capture them.

6.6 *Purely functional imperative programming*

In the preceding sections, we were writing functions that followed a definite pattern. We'd run a state action, assign its result to a `val`, then run another state action that used that `val`, assign its result to another `val`, and so on. It looks a lot like *imperative* programming.

In the imperative programming paradigm, a program is a sequence of statements where each statement may modify the program state. That's exactly what we've been doing, except that our "statements" are really `State` actions, which are really functions. As functions, they read the current program state simply by receiving it in their argument, and they write to the program state simply by returning a value.

Aren't imperative and functional programming opposites?

Absolutely not. Remember, functional programming is simply programming without side effects. Imperative programming is about programming with statements that modify some program state, and as we've seen, it's entirely reasonable to maintain state without side effects.

Functional programming has excellent support for writing imperative programs, with the added benefit that such programs can be reasoned about equationally because they're referentially transparent. We'll have much more to say about equational reasoning about programs in part 2, and imperative programs in particular in parts 3 and 4.

We implemented some combinators like `map`, `map2`, and ultimately `flatMap` to handle the propagation of the state from one statement to the next. But in doing so, we seem to have lost a bit of the imperative mood.

Consider as an example the following (which assumes that we've made `Rand[A]` a type alias for `State[RNG, A]`):

```
val ns: Rand[List[Int]] =
  int.flatMap(x =>
    int.flatMap(y =>
      ints(x).map(xs =>
        xs.map(_ % y)))
```

int is a value of type `Rand[Int]` that generates a single random integer.

ints(x) generates a list of length **x**.

Replaces every element in the list with its remainder when divided by **y**.

It's not clear what's going on here. But since we have `map` and `flatMap` defined, we can use a for-comprehension to recover the imperative style:

```
val ns: Rand[List[Int]] = for {
  x <- int
  y <- int
  xs <- ints(x)
} yield xs.map(_ % y)
```

Generates an integer **x**.

Generates another integer **y**.

Generates a list **xs** of length **x**.

Returns the list **xs** with each element replaced with its remainder when divided by **y**.

This code is much easier to read (and write), and it looks like what it is—an imperative program that maintains some state. But it's *the same code*. We get the next `Int` and assign it to `x`, get the next `Int` after that and assign it to `y`, then generate a list of length `x`, and finally return the list with all of its elements modulo `y`.

To facilitate this kind of imperative programming with for-comprehensions (or `flatMap`s), we really only need two primitive `State` combinators—one for reading the state and one for writing the state. If we imagine that we have a combinator `get` for

getting the current state, and a combinator `set` for setting a new state, we could implement a combinator that can modify the state in arbitrary ways:

```
def modify[S](f: S => S): State[S, Unit] = for {
  s <- get
  _ <- set(f(s))
} yield ()
```

Gets the current state and assigns it to `s`

Sets the new state to `f` applied to `s`

This method returns a `State` action that modifies the incoming state by the function `f`. It yields `Unit` to indicate that it doesn't have a return value other than the state.

What would the `get` and `set` actions look like? They're exceedingly simple. The `get` action simply passes the incoming state along and returns it as the value:

```
def get[S]: State[S, S] = State(s => (s, s))
```

The `set` action is constructed with a new state `s`. The resulting action ignores the incoming state, replaces it with the new state, and returns `()` instead of a meaningful value:

```
def set[S](s: S): State[S, Unit] = State(_ => ((), s))
```

These two simple actions, together with the `State` combinators that we wrote—`unit`, `map`, `map2`, and `flatMap`—are all the tools we need to implement any kind of state machine or stateful program in a purely functional way.



EXERCISE 6.11

Hard: To gain experience with the use of `State`, implement a finite state automaton that models a simple candy dispenser. The machine has two types of input: you can insert a coin, or you can turn the knob to dispense candy. It can be in one of two states: locked or unlocked. It also tracks how many candies are left and how many coins it contains.

```
sealed trait Input
case object Coin extends Input
case object Turn extends Input

case class Machine(locked: Boolean, candies: Int, coins: Int)
```

The rules of the machine are as follows:

- Inserting a coin into a locked machine will cause it to unlock if there's any candy left.
- Turning the knob on an unlocked machine will cause it to dispense candy and become locked.
- Turning the knob on a locked machine or inserting a coin into an unlocked machine does nothing.
- A machine that's out of candy ignores all inputs.